

Access Modifier:

v.length
v.setlength

int length;

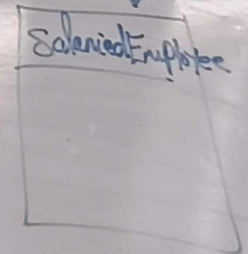
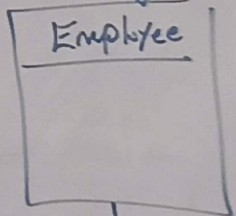
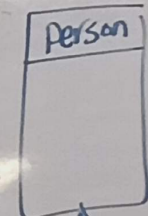
private
public
protected

default case

Package
Shape
Rectangle
Circle

Access Modifier	Same Package		different Package	
	Same class	other classes	Subclass	other classes
private	✓	X	X	X
public	✓	✓	✓	✓
protected	✓	✓	✓	X
default	✓	✓	X	X

Sub class
عنوان ليد
حاجه
في Package



Abstract class:
 - Cannot be instantiated
 - may be contain abstract Method
 - Subclass must override the abstract method

Syntax:

```
public abstract class class name
{
    // ...
}
```

Abstract Method:
 - Method without body

```
Public abstract class Employee extends Person
{
    protected double Salary;
    protected String rank;
    protected String Job;

    Public Employee (
        String name, int age,
        String address, String nationality,
        double salary, String rank,
        String Job
    )
    {
        Super (name, age, address, nationality);
        this.Salary = salary;
        this.rank = rank;
        this.Job = Job;
    }

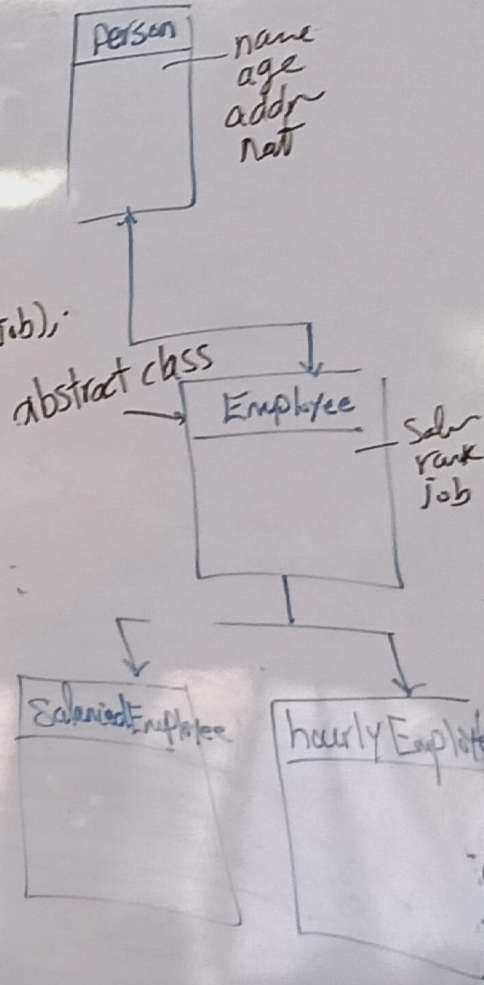
    Public void setsalary (double salary)
    {
        this.Salary = salary;
    }

    Public double getsalary ( )
    {
        return Salary;
    }
}
```

```
Public class SalariesEmployee extends Employee
{
    private double bonus;
    private double deductions;

    Public SalariesEmployee (
        String name, int age, String address,
        String nationality, double salary,
        String rank, String Job,
        double bonus, double deductions
    )
    {
        Super (name, age, address, nationality, salary, rank, Job);
        this.bonus = bonus;
        this.deductions = deductions;
    }

    @Override
    Public double getsalary ( )
    {
        return salary + bonus - deductions;
    }
}
```



Abstract class:

- Cannot be instantiated
- may be contain abstract Method
- Subclass must override the abstract method

Syntax:

```
public abstract class class name  
{  
          
}
```

Abstract Method:

- Method without body
- ```
public abstract double getsalary();
```

```
Public abstract class Employee extends Person
{
 protected double Salary;
 protected String rank;
 protected String Job;
}
```

```
Public Employee()
{
 String name, int age,
 String address, String nationality;
 double salary, String rank,
 String Job
}
```

```
Super(name, age, address, nationality);
this.salary = salary;
this.rank = rank;
this.Job = Job;
```

```
Public void setsalary(double salary)
{
 this.salary = salary;
}
```

```
Public abstract double getsalary();
```

```
Public class SalariedEmployee extends Employee
```

```
{
 private double bonus;
 private double deductions;
 public SalariedEmployee(
 String name, int age, String address,
 String nationality, double salary,
 String rank, String Job,
 double bonus, double deductions)
 {
 super(name, age, address, nationality, salary, rank, Job);
 this.bonus = bonus;
 this.deductions = deductions;
 }
}
```

```
Public double getsalary()
{
 return salary + bonus - deductions;
}
```

```
Public class test
```

```
{
 public static void main(String[] args)
 {
 Employee E = new Employee();
 SalariedEmployee S = new SalariedEmployee();
 }
}
```

override the abstract method

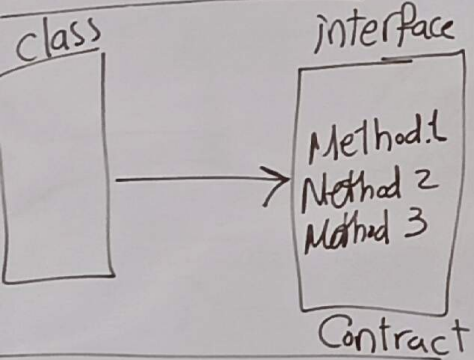
abstract class

abstr

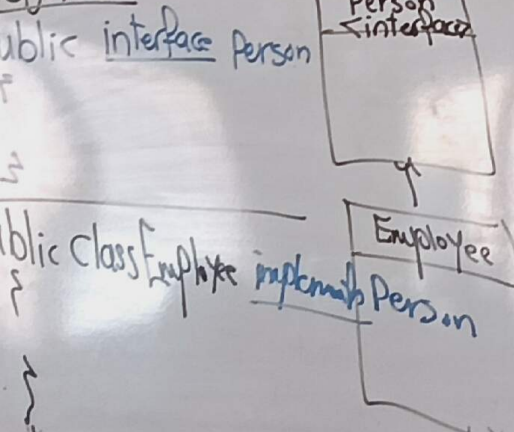
class

## interface

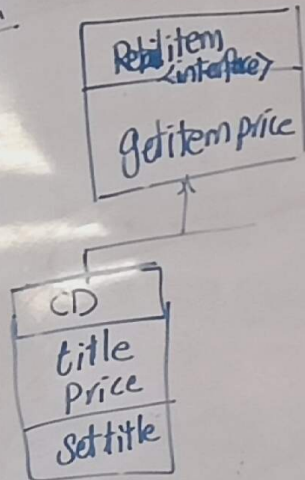
Cannot be instantiated  
All methods  $\Rightarrow$  abstract



## Syntax



Ex:



Super  $\rightarrow$  new Sub

Public interface RetainItem

```
{
 public double getItemPrice()
}
```

Public class CD implements RetainItem

```
{
 private String title;
 private double Price;
 public CD()
 {
 }
 public CD(String title, double price)
 {
 this.title = title;
 this.price = price;
 }
 public void setTitle(String title)
 {
 this.title = title;
 }
 @Override
 public double getItemPrice()
 {
 return Price;
 }
}
```

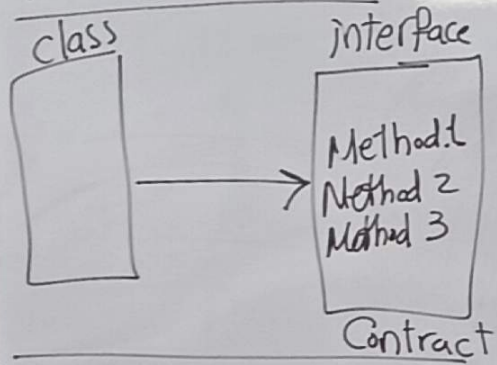
Public class test

```
{
 public static void main(String[] args)
 {
 RetainItem r = new RetainItem ();
 CD c = new CD ();
 }
}
```



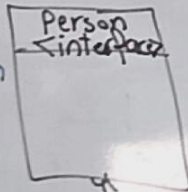
## interface

- Cannot be instantiated
- All methods  $\Rightarrow$  abstract

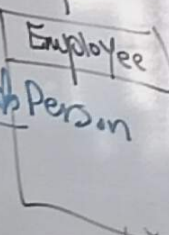


## Syntax

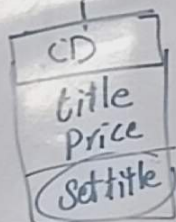
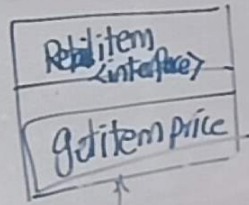
Public interface Person



Public class Employee implements Person

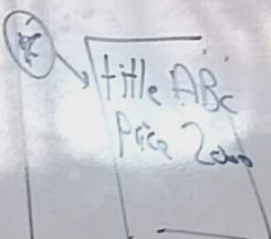


Ex:



Super  $\rightarrow$  new Sub

Method



Public interface RetailItem

```
{
 public double getItemPrice()
}
```

Public class CD implements RetailItem

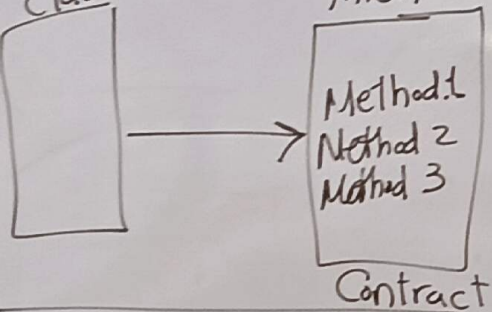
```
{
 private String title;
 private double price;
 public CD()
 {
 title = "ABC";
 price = 2000;
 }
 public CD(String title, double price)
 {
 this.title = title;
 this.price = price;
 }
 public void setTitle(String title)
 {
 this.title = title;
 }
 @Override
 public double getItemPrice()
 {
 return price;
 }
}
```

Public class test

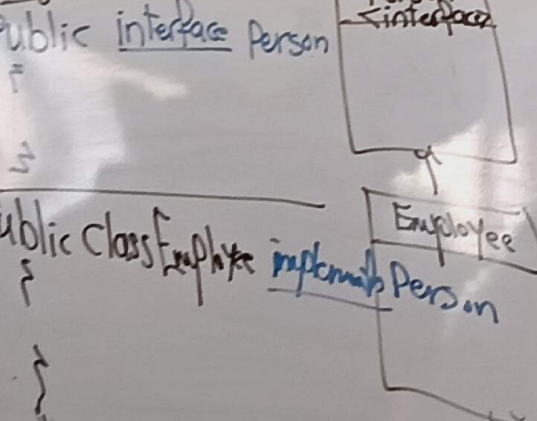
```
{
 public static void main(String[] args)
 {
 RetailItem r = new RetailItem();
 CD c = new CD();
 CD c = new CD("ABC", 2000);
 c.setTitle("XYZ");
 Sy = (c.getItemPrice());
 Sy = (r.getItemPrice());
 r.setTitle("XYZ");
 }
}
```

## interface

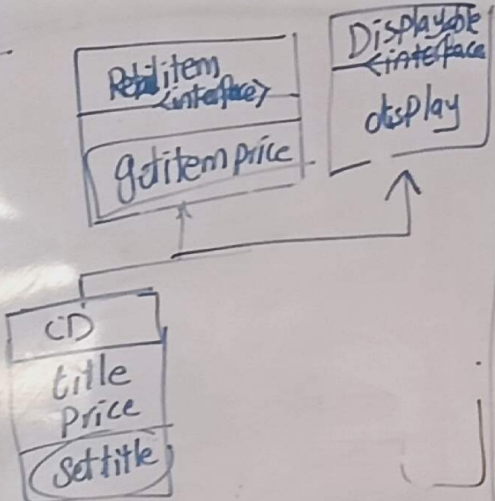
- Cannot be instantiated
- All methods  $\Rightarrow$  abstract
- Data fields  $\Rightarrow$  public, static, final
- Class



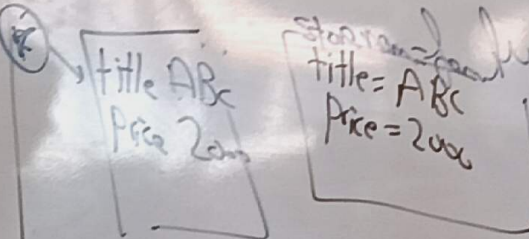
## Syntax



Ex:



Super  $\rightarrow$  = new Sub  
 ↓  
 Method



```

public interface RetailItem {
 String storeName = "Jaluti";
 public double getTitlePrice();
}

```

```

public interface Displayable {
 public void display();
}

```

```

public class CD implements RetailItem, Displayable {
 private String title;
 private double price;

 public CD() {
 // ABC 2000
 }

 public CD(String title, double price) {
 this.title = title;
 this.price = price;
 }

 public void setTitle(String title) {
 this.title = title;
 }

 public double getTitlePrice() {
 return price;
 }

 public void display() {
 System.out.println("Store name: " + storeName);
 System.out.println("title: " + title);
 System.out.println("price: " + price);
 }
}

```

```

public static void main(String[] args) {
 RetailItem r = new RetailItem();
 CD c = new CD();
 CD c = new RetailItem();
 CD c = new CD("ABC", 2000);
 c.setTitle("XYZ");
 System.out.println(c.getTitlePrice());
 RetailItem r = new CD("ABC", 2000);
 System.out.println(r.getTitlePrice());
 r.setTitle("XYZ");
 CD c = new CD("ABC", 2000);
 c.setTitle("XYZ");
}

```